

CodePDE: An Inference Framework for LLM-driven PDE Solver Generation

Journal Club

Introduction

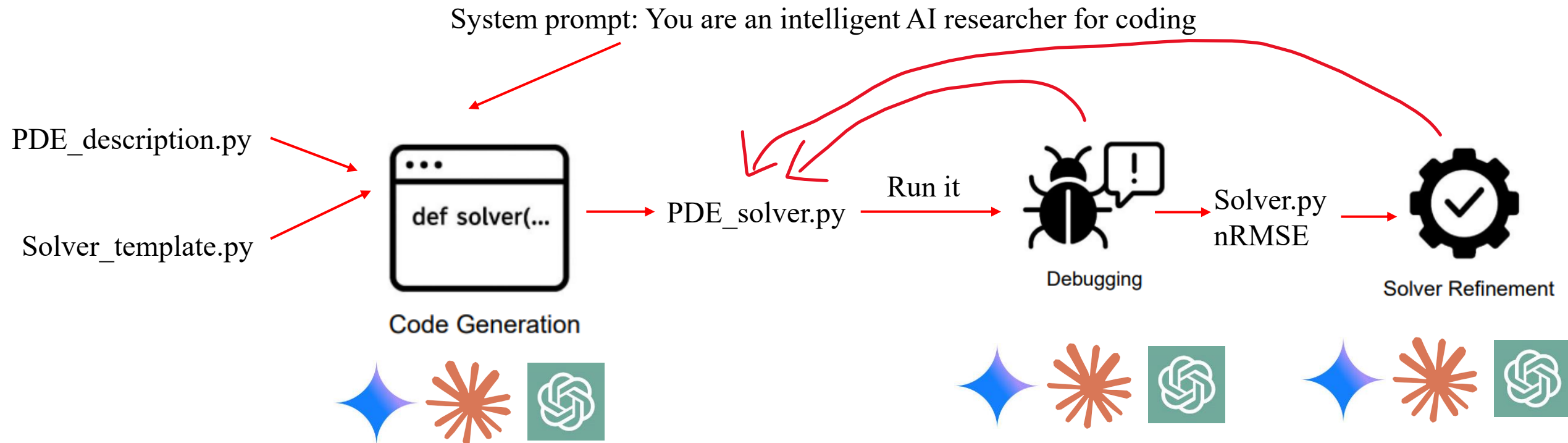
- **Partial differential equations** (PDEs) are fundamental to modeling physical systems, yet solving them remains a complex challenge.
- Traditional **numerical solvers** rely on expert knowledge to implement and are computationally expensive, while **neural-network-based solvers** require large training datasets and often lack interpretability.
- Here it used a novel **multi-agent LLM-driven** PDE solver.

Introduction – Existing LLM PDE solver

- **CodePDE**: multi-agent LLM-driven PDE solver
- **OpInf-LLM**: data driven method (Operator Inference) combined with LLM.
- **MCP-SIM**: A self-correcting multi-agent LLM framework for physics simulation.
- **PINNsAgent**: Physics Neural Network combined with LLM.

Methods – Multi-Agent

- LLM only prompt: Given PDE... and Initial condition..., solve it... . Try to avoid bugs... Return the final solution as csv...
- LLM with multi agent:



Methods – system prompt

System Prompt

You are an intelligent AI researcher for coding, numerical algorithms, and scientific computing.
Your goal is to conduct cutting-edge research in the field of PDE solving by leveraging and creatively improving existing algorithms to maximize performances based on feedbacks.
Follow the user's requirements carefully and make sure you understand them.
Always document your code as comments to explain the reason behind them.
Always use Markdown cells to present your code.

Methods – pde description prompt

- In Latex

Your task is to solve a partial differential equation (PDE) using Python. The PDE is the Burgers equation with the periodic boundary condition, given by

$$\begin{cases} \partial_t u(x, t) + \partial_x \left(\frac{u^2(x, t)}{2} \right) = \nu \partial_{xx} u(x, t), & x \in (0, 1), t \in (0, 1] \\ u(x, 0) = u_0(x), & x \in (0, 1) \end{cases},$$

where ν is a constant representing the viscosity.

Your task is to solve a partial differential equation (PDE) using Python in batch mode. The PDE is the 1D compressible Navier-Stokes equations, given by

$$\begin{cases} \partial_t \rho + \partial_x(\rho v) = 0 \\ \rho(\partial_t v + v \partial_x v) = -\partial_x p + \eta \partial_{xx} v + (\zeta + \eta/3) \partial_x(\partial_x v) \\ \partial_t \left[\epsilon + \frac{\rho v^2}{2} \right] + \partial_x \left[\left(\epsilon + p + \frac{\rho v^2}{2} \right) v - v \sigma' \right] = 0 \end{cases}$$

where ρ is the mass density, v is the velocity, p is the gas pressure, $\epsilon = p/(\Gamma - 1)$ is the internal energy with $\Gamma = 5/3$, $\sigma' = (\zeta + \frac{4}{3}\eta)\partial_x v$ is the viscous stress tensor, and η, ζ are the shear and bulk viscosity coefficients, respectively. In our task, we assume periodic boundary conditions. The spatial domain is $\Omega = [-1, 1]$.

Given the discretization of the initial velocity, density, pressure, each of shape [batch_size, N] where N is the number of spatial points, you need to implement a solver to predict the state variables for the specified subsequent time steps ($t = t_1, \dots, t_T$). The solver outputs velocity, density, pressure, each of shape [batch_size, T+1, N] (with the initial time frame and the subsequent steps). Note that although the required time steps are specified, you should consider using smaller time steps internally to obtain more stable simulation.

In particular, your code should be tailored to the case where $\eta = \zeta = 0.1$, i.e., optimizing it particularly for this use case.

Methods – solver template prompt

```
import numpy as np

def solver(Vx0, density0, pressure0, t_coordinate, eta, zeta):
    """Solves the 1D compressible Navier-Stokes for all times in t_coordinate.

    Args:
        Vx0 (np.ndarray): Initial velocity of shape [batch_size, N]
            where N is the number of evenly spaced spatial points.
        density0 (np.ndarray): Initial density [batch_size, N].
        pressure0 (np.ndarray): Initial pressure [batch_size, N].
        t_coordinate (np.ndarray): Time coordinates of shape [T+1].
            It begins with t_0=0 and follows the time steps t_1, ..., t_T.
        eta (float): The shear viscosity coefficients in the equation.
        rho (float): The bulk viscosity coefficients in the equation.

    Returns:
        Vx_pred (np.ndarray): Shape [batch_size, T+1, N].
            The first timeframe is identical to Vx0.
            The subsequent timeframes are the solutions at the corresponding
            time steps.
        density_pred (np.ndarray): Shape [batch_size, T+1, N].
        pressure_pred (np.ndarray): Shape [batch_size, T+1, N].
    """
    # TODO: Implement the solver for 1D compressible Navier-Stokes equations
    # Hints:
    # 1. Consider using PyTorch or JAX with GPU acceleration
    # 2. Remember to handle data types and device placement appropriately
    return Vx_pred, density_pred, pressure_pred
```

Methods – debug prompt

Example Debugging Prompt

Thank you for your implementation! When running the code, I got the following output and error message:

Code output: {code_output}

Error message: {error_message}

Can you think step-by-step to identify the root cause of the error and provide a solution to fix it?

Please provide a detailed explanation of the error and the solution you propose. You can refer to the code implementation you provided earlier and analyze the error message to identify the issue.

Your response should be in the following format:

[Your rationale for debugging (think step-by-step)]

```
'''python  
[Your bug-free implementation]  
'''
```


Methods – refinement prompt

Example Refinement Prompt

Your task is to solve a partial differential equation (PDE) using Python in batch mode.

`{pde_description}`

You will be improving the following existing code samples. The code samples are provided below:

`{code_samples}`

Your tasks are:

1. Understand the above code samples. Compare their techniques and performances.
2. Identify the parts that could potentially be improved.
3. Plan on how you can improve the function.
4. Improve the function.

The goal is to get a very low nRMSE (normalized RMSE) and make the code as fast as possible.

You must analyze the implementation and test results of the examples provided in the code template, and think about how you can improve them to reduce the nRMSE.

If the RMSE is much higher than $1e-2$ or becomes Nan, it is likely that there is a bug in the implementation and you must debug it or think about completely different approaches.

If the running time is much longer than 600s, you must prioritize making it more efficient.

The convergence rate is the empirical order of convergence with respect to spatial resolution. It is also a good indicator of the performance of the algorithm which you may consider.

You can implement auxiliary functions or add additional arguments to the function if needed. You can use PyTorch or JAX with GPU acceleration. You can consider known techniques and analyze their effectiveness based on existing results. You should also consider combining existing techniques or even developing new techniques since you are doing cutting-edge research.

Your generated code will be executed to evaluated. Make sure your 'solver' function runs correctly and efficiently. You must use print statements for to keep track of intermediate results, but do not print too much information. Those output will be useful for future code improvement and/or debugging.

Your output must follow the following structure:

1. Summary of the existing implementation along with their performances. Identify what techniques work well, what could be buggy (due to high error or Nan), and what could be further improved.
2. Rationale for the new implementation (think step-by-step) based on the summary of the existing implementation.
3. Your python implementation (modularized with appropriate auxiliary functions):

```
'''python
[Your improved implementation]
'''
```

Evaluation data – PDEbench

- A benchmark designed to evaluate how well machine learning models can solve or predict PDEs.

Table 1: Summary of PDEBENCH’s datasets with their respective number of spatial dimensions N_d , time dependency, spatial resolution N_s , temporal resolution N_t , and number of samples generated.

PDE	N_d	Time	N_s	N_t	Number of samples
advection	1	yes	1 024	200	10 000
Burgers’	1	yes	1 024	200	10 000
diffusion-reaction	1	yes	1 024	200	10 000
diffusion-reaction	2	yes	128×128	100	1000
diffusion-sorption	1	yes	1 024	100	10 000
compressible Navier-Stokes	1	yes	1 024	100	10 000
compressible Navier-Stokes	2	yes	512×512	21	1000
compressible Navier-Stokes	3	yes	$128 \times 128 \times 128$	21	100
incompressible Navier-Stokes	2	yes	256×256	1000	1000
Darcy flow	2	no	128×128	–	10 000
shallow-water	2	yes	128×128	100	1000

Result - nRMSE

$$\text{nRMSE} = \frac{1}{S} \sum_{s=1}^S \frac{\|\mathbf{u}^{(s)}(\mathbf{x}, t) - \hat{\mathbf{u}}^{(s)}(\mathbf{x}, t)\|_2}{\|\mathbf{u}^{(s)}(\mathbf{x}, t)\|_2}$$

- S: number of PDE example

Table 1: **Normalized RMSE comparisons.** “Geom. Mean” refers to the geometric mean of nRMSE values. Neural Network baseline results are from (Takamoto et al., 2022; Li et al., 2022b; Ye et al., 2024; Shen et al., 2023; 2024b). The best results of the “Reasoning + Debugging” and the “Reasoning + Debugging + Refinement” settings for each problem are highlighted in gray cells.

nRMSE ($\downarrow$)	Advection	Burgers	React-Diff	CNS	Darcy	Geom. Mean
Reference Numerical Solvers						
	1.03×10^{-3}	3.55×10^{-4}	2.29×10^{-3}	1.89×10^{-2}	4.80×10^{-3}	2.38×10^{-3}
PDE Solving Softwares						
Dedalus	6.20×10^{-3}	2.78×10^{-3}	5.20×10^{-2}	2.01×10^{-2}	—	—
PyPDE	2.03×10^{-3}	5.92×10^{-4}	2.21×10^{-1}	2.52×10^{-1}	—	—
Neural Network & Foundation Model baselines						
U-Net	5.00×10^{-2}	2.20×10^{-1}	6.00×10^{-3}	3.60×10^{-1}	—	—
FNO	7.70×10^{-3}	7.80×10^{-3}	1.40×10^{-3}	9.50×10^{-2}	9.80×10^{-3}	9.52×10^{-3}
PINN	7.80×10^{-3}	8.50×10^{-1}	8.00×10^{-2}	—	—	—
ORCA	9.80×10^{-3}	1.20×10^{-2}	3.00×10^{-3}	6.20×10^{-2}	—	—
PDEformer	4.30×10^{-3}	1.46×10^{-2}	—	—	—	—
UPS	2.20×10^{-3}	3.73×10^{-2}	5.57×10^{-2}	4.50×10^{-3}	—	—
CodePDE: Reasoning + Debugging (best of 32)						
Gemini 2.0 Flash	1.14×10^{-3}	2.97×10^{-4}	2.19×10^{-1}	4.50×10^{-2}	4.80×10^{-3}	6.92×10^{-3}
Gemini 2.0 Thinking	5.54×10^{-3}	3.21×10^{-4}	2.19×10^{-1}	7.56×10^{-2}	4.80×10^{-3}	1.07×10^{-2}
Gemini 2.5 Pro	1.01×10^{-3}	1.23×10^{-4}	1.49×10^{-1}	7.39×10^{-2}	4.89×10^{-3}	5.82×10^{-3}
Qwen-2.5-Max	4.97×10^{-3}	1.35×10^{-3}	9.57×10^{-2}	2.36×10^{-1}	6.76×10^{-1}	4.00×10^{-2}
QwQ-Plus	1.03×10^{-3}	3.05×10^{-4}	2.20×10^{-1}	7.43×10^{-2}	4.80×10^{-3}	7.55×10^{-3}
Claude-3.5-haiku	3.70×10^{-3}	3.23×10^{-4}	2.20×10^{-1}	2.34×10^{-1}	1.00×10^0	3.61×10^{-2}
Claude-3.7-sonnet	1.32×10^{-3}	2.59×10^{-4}	5.19×10^{-2}	4.26×10^{-2}	4.80×10^{-3}	5.15×10^{-3}
DeepSeek-V3	7.37×10^{-3}	6.02×10^{-4}	2.18×10^{-1}	2.41×10^{-2}	1.00×10^0	2.98×10^{-2}
DeepSeek-R1	1.05×10^{-3}	2.76×10^{-4}	2.20×10^{-1}	7.46×10^{-2}	4.80×10^{-3}	7.44×10^{-3}
GPT-4o	1.55×10^{-3}	3.69×10^{-4}	1.99×10^{-2}	1.81×10^{-1}	7.67×10^{-1}	1.74×10^{-2}
GPT-4.1	1.50×10^{-3}	3.63×10^{-4}	1.44×10^{-1}	5.20×10^{-2}	4.88×10^{-3}	7.23×10^{-3}
o3	9.74×10^{-4}	3.69×10^{-4}	1.98×10^{-1}	2.80×10^{-2}	4.92×10^{-3}	6.28×10^{-3}
CodePDE: Reasoning + Debugging + Refinement (best of 12)						
Gemini 2.0 Flash	8.24×10^{-4}	1.06×10^{-4}	1.76×10^{-2}	1.72×10^{-2}	4.78×10^{-3}	2.63×10^{-3}
Gemini 2.0 Thinking	9.74×10^{-4}	2.70×10^{-4}	1.74×10^{-2}	2.41×10^{-2}	4.80×10^{-3}	3.51×10^{-3}
Gemini 2.5 Pro	9.74×10^{-4}	3.15×10^{-4}	9.94×10^{-2}	2.77×10^{-2}	4.92×10^{-3}	5.29×10^{-3}
Qwen-2.5-Max	9.74×10^{-4}	2.60×10^{-4}	9.13×10^{-2}	1.49×10^{-2}	4.80×10^{-3}	4.40×10^{-3}
QwQ-Plus	9.74×10^{-4}	1.07×10^{-4}	5.19×10^{-2}	1.50×10^{-2}	4.80×10^{-3}	3.29×10^{-3}
Claude-3.5-haiku	1.01×10^{-3}	2.60×10^{-4}	5.19×10^{-2}	1.06×10^{-1}	1.92×10^{-1}	1.22×10^{-2}
Claude-3.7-sonnet	9.74×10^{-4}	2.60×10^{-4}	5.19×10^{-2}	2.72×10^{-2}	4.83×10^{-3}	4.44×10^{-3}
DeepSeek-V3	9.74×10^{-4}	2.59×10^{-4}	1.74×10^{-2}	1.56×10^{-2}	4.80×10^{-3}	3.18×10^{-3}
DeepSeek-R1	9.74×10^{-4}	2.59×10^{-4}	1.43×10^{-1}	1.52×10^{-2}	4.80×10^{-3}	4.83×10^{-3}
GPT-4o	9.74×10^{-4}	2.59×10^{-4}	1.76×10^{-2}	2.41×10^{-2}	4.80×10^{-3}	3.48×10^{-3}
GPT-4.1	1.01×10^{-3}	3.15×10^{-4}	1.44×10^{-1}	1.53×10^{-2}	4.88×10^{-3}	5.08×10^{-3}
o3	1.01×10^{-3}	3.15×10^{-4}	2.01×10^{-1}	1.31×10^{-2}	4.88×10^{-3}	5.27×10^{-3}

Result – bug-free rate

- Bug-free rates tested across 32 independent trails of LLM coding.

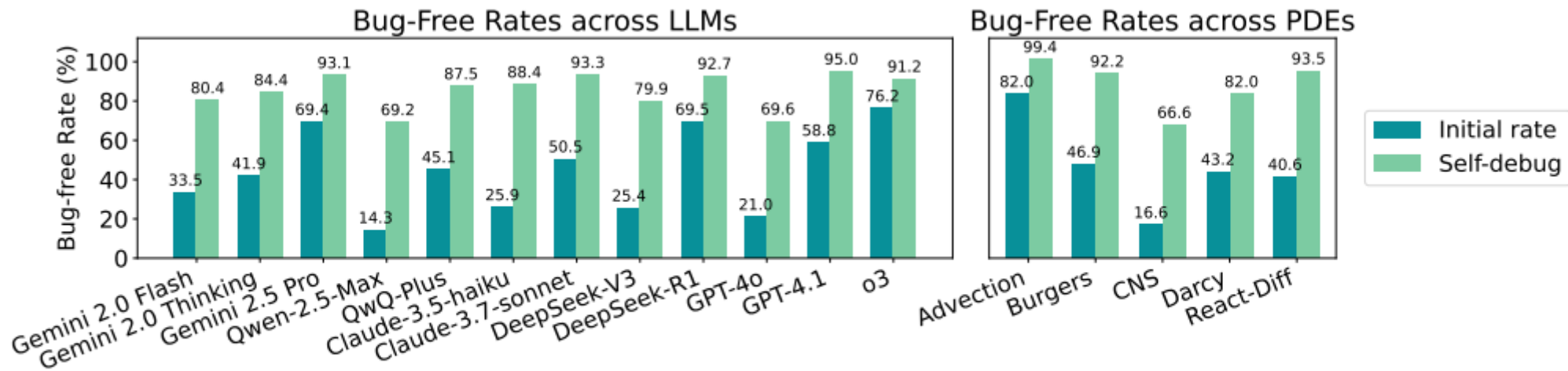


Figure 2: Bug-free rate before and after introducing automated debugging. **Left:** Averaged across all five PDE datasets for each model. **Right:** Averaged across all LLMs for each PDE family.

Result – test time scaling

- Test time scaling tested across 32 independent trails of LLM coding.

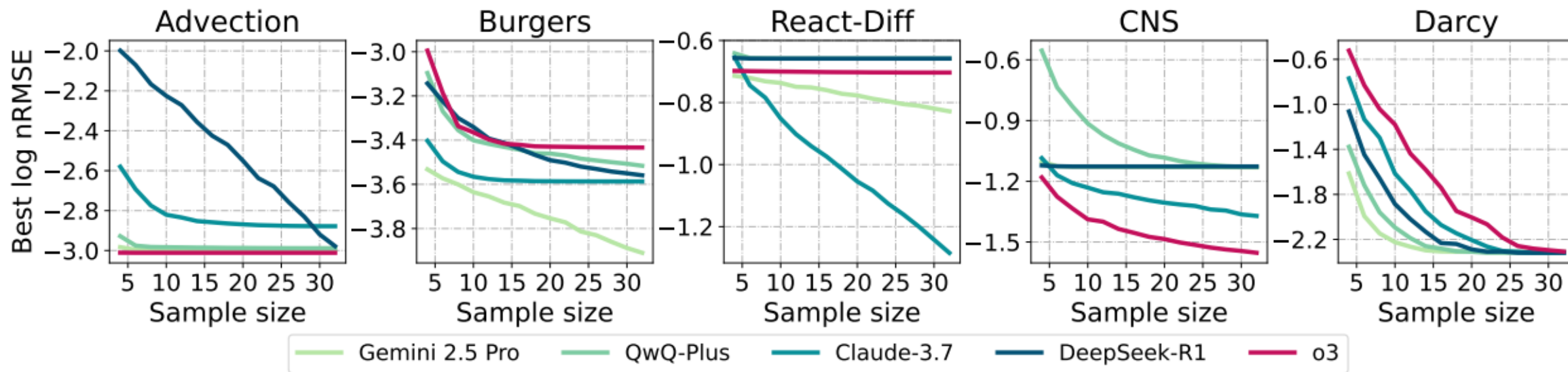
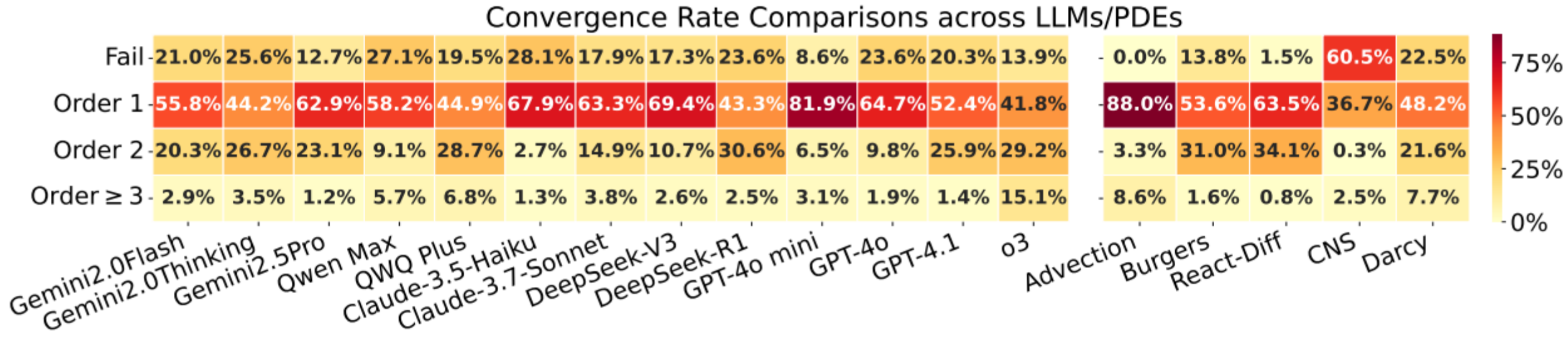


Figure 3: **Smoothed test-time scaling curves** for representative models on each PDE family. We increase n in the best-of- n sampling to study test-time scaling performance.

Result – Convergence rate

- A solver is considered convergent if the nRMSE decreases as the grid is refined. The author defined four level: fail (no convergence or very slow convergence), order 1 2 3 (faster convergence)



Result – Ablation test

- Ablation test: testing performance if deleting some parts of the framework. (scale means multiple times trails of LLM coding)

Table 2: **Ablation study on CodePDE and evaluation of existing agentic workflows.** All results are based on Claude-3.7-sonnet for a consistent comparison. Full results on 16 LLMs are provided in Appendix Table 4.

nRMSE ($\downarrow$)	Advection	Burgers	React-Diff	CNS	Darcy	Geom. Mean
CodePDE Ablation						
CodePDE	9.74×10^{-4}	2.60×10^{-4}	5.19×10^{-2}	2.72×10^{-2}	4.83×10^{-3}	4.44×10^{-3}
w/o refine	1.32×10^{-3}	2.59×10^{-4}	5.19×10^{-2}	4.26×10^{-2}	4.80×10^{-3}	5.15×10^{-3}
w/o refine+scale	1.37×10^{-3}	4.60×10^{-3}	4.26×10^{-1}	3.02×10^{-1}	6.06×10^{-1}	8.68×10^{-2}
w/o refine+scale+debug	2.16×10^{-3}	1.00×10^{-2}	7.12×10^{-1}	6.62×10^{-1}	7.16×10^{-1}	1.49×10^{-1}
Evaluation of Existing Agentic Workflows						
FunSearch	9.84×10^{-4}	2.59×10^{-4}	5.12×10^{-2}	3.79×10^{-2}	4.80×10^{-3}	4.73×10^{-3}
AIDE	1.03×10^{-3}	1.05×10^{-4}	5.07×10^{-2}	5.77×10^{-2}	4.78×10^{-3}	4.33×10^{-3}

Result – Integration schemes employed

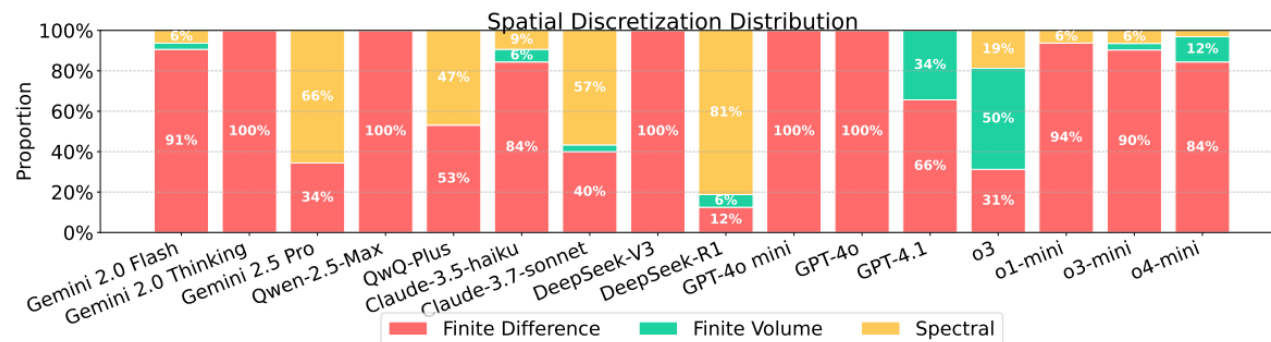


Figure 5: Distributions of the spatial discretization schemes employed by each LLM.

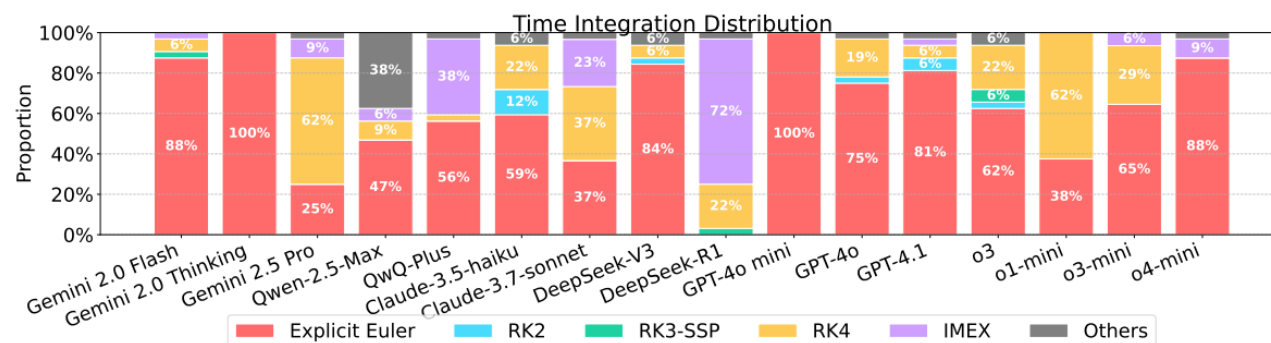


Figure 6: Distributions of the time integration schemes employed by each LLM. “RK2”, “RK3-SSP”, “RK4”, and “IMEX” refers to Runge-Kutta 2nd order, Runge-Kutta 3rd order (strong stability preserving), Runge-Kutta 4th order, and Implicit-Explicit schemes, respectively.

Result – Code Execution t

- Running time of pde solver, not including the time to generate LLM responses.
- Running time on 100 instances for a specific pde family

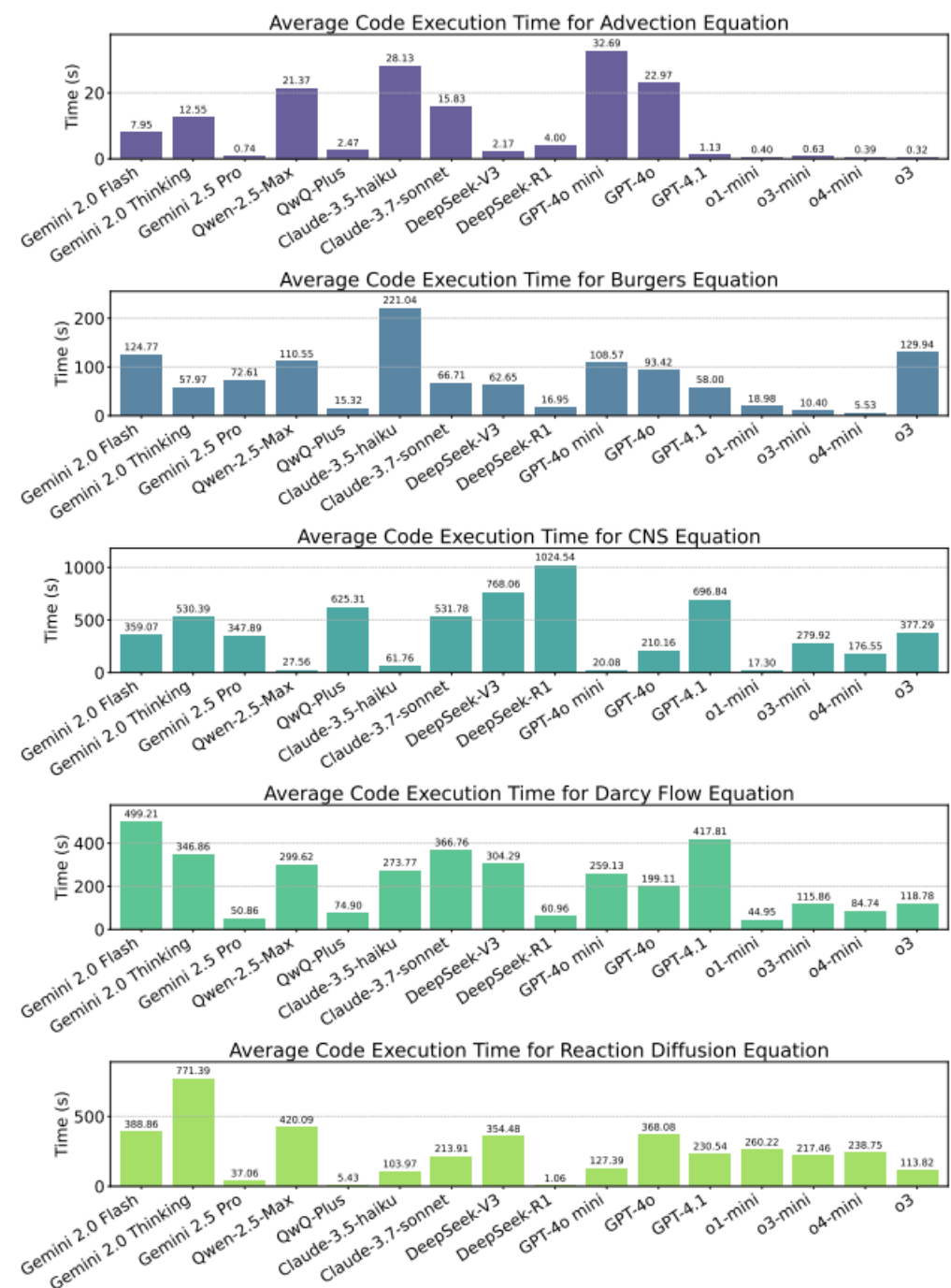


Figure 13: The average code execution time for each LLM on each PDE family.

Conclusion

- A structured inference framework with the combination of automated debugging, refinement, and test-time scaling is essential to solve PDEs with LLM.
- Pure **prompt engineering** to generate reliable PDE solver is enough, without requiring any post or pre **training** of LLM.